



Energy efficiency for edge multimedia elastic applications

Alessandro Carrega¹ · Giancarlo Portomauro² · Matteo Repetto¹  · Giorgio Robino¹

Received: 11 September 2018 / Revised: 19 November 2018 / Accepted: 7 December 2018 /

Published online: 2 May 2019

© Springer Science+Business Media, LLC, part of Springer Nature 2019

Abstract

With the rise of edge computing paradigms, multimedia applications will have to tackle unprecedented management issues, pursuing an optimal balance between performance, Quality of Service (QoS), and power consumption. In this paper, we investigate a novel paradigm to deploy multimedia elastic applications at the edge in a very energy-efficient manner. Our approach is based on pre-provisioning virtual resources that remain “frozen” until the application scales out. Frozen resources are treated in a special way by the infrastructure, leveraging aggressive power-saving mechanisms that keep negligible their impact on energy consumption and performance. We report extensive measurements on QoS and power consumption that we carried out in a real testbed, which is the first working implementation of the proposed paradigm. Our work shows how resource utilization and performance can be increased by leveraging SDN technologies and conscious setting of cloud parameters. We investigate the trade-off between performance and power consumption (i.e., energy efficiency), in relation to different consolidation strategies. Finally, we measure power consumption and estimate energy saving for an elastic video transcoding application deployed at the network edge.

Keywords Energy-efficiency · Quality of service · Elastic transcoding · Edge computing

✉ Alessandro Carrega
alessandro.carrega@cnit.it

Giancarlo Portomauro
giancarlo.portomauro@edu.unige.it

Matteo Repetto
matteo.repetto@cnit.it

Giorgio Robino
giorgio.robino@cnit.it

¹ CNIT – S3ITILab, Via all’Opera Pia 13, 16145 Genoa, Italy

² University of Genoa – TNT Lab, Via all’Opera Pia 13, 16145 Genoa, Italy

1 Introduction

Edge computing is a recent cloud paradigm that deploys computing and storage resources at the network boundary, hence allowing applications to be moved closer to the user. This approach greatly reduces latency and saves bandwidth in long-distance links, boosting the deployment of mission-critical and interactive applications. However, it also brings additional management issues, especially in wireless networks, due to user mobility and the need to migrate applications.

Among others, multimedia applications are natural candidates to exploit the new paradigm. Typical multimedia applications deployed at the network edge (e.g., video streaming and transcoding, gaming, augmented reality) are expected to experience far more variability in usage patterns than what happens in the cloud, due to uneven and variable user distribution. From a technical perspective, user mobility is the most challenging aspect [6], since it has a direct impact on service continuity and the overall quality of experience (QoE). The overprovisioning model (i.e., acquiring resources according to the maximum load) has been the common ‘least-effort’ relief for long, but it is no more acceptable, because it leads to inefficient utilization of resources and large energy waste.

The design of elastic applications is a cost-effective mean to tackle large variations in workload. In the Infrastructure-as-a-Service (IaaS) cloud model, an elastic application scales vertically, i.e., increasing or decreasing the size of Virtual Machines (CPU, RAM, bandwidth, storage), or horizontally, i.e., adding or removing VMs. Elastic applications are easy to implement when processing can be parallelized (this is the typical case for video transcoding), but the time to scale in or out is not negligible: as a matter of fact, it usually takes from minutes to days to have a VM ready, depending on the automation level, which clearly does not fulfill the strict latency requirements for interactive applications as streaming, transcoding and gaming.

In this paper, we investigate a new way to efficiently deploy multimedia elastic applications at the edge. The specific objective is to reduce the time to scale in and out the applications, while reducing the overall energy consumption of the infrastructure. Our approach leverages power-saving mechanisms already available in the hardware; we pre-provision computing resources with a negligible energy footprint until they are really used. More in details, our framework creates VMs in advance, keeps them in special “frozen” states, and makes them quickly available when scaling is needed.

The main contribution of our work is a better balance between performance, Quality of Service, and energy consumption for elastic multimedia applications at the edge, which improves the cost-revenue ratio for service providers. Despite a large number of works on energy efficiency and workload consolidation is already present in the literature, the trade-off between performance and energy is barely evaluated, especially in real testbeds. This paper extends our previous work which already designed and implemented an energy-efficient infrastructure for edge computing, motivated by the substantial lack of power management and QoS capabilities in existing cloud management software [9]. That work described the overall infrastructure design and implementation, but only provided very preliminary experimental investigation. Now, we conduct thorough investigation of QoS, energy consumption, and their relationship in a real testbed for an elastic transcoding application.

The rest of the paper is organized as follows. Section 2 discusses related work and highlights the main innovations of our approach. Section 3 briefly describes our

energy efficiency paradigm and its implementation. Section 4 describes the experimental testbed used for performance evaluation. We report a number of experiments on performance and QoS in Section 5, and we correlate performance with power consumption in Section 6. Application to an elastic transcoding application and related performance evaluation are reported in Section 7. Finally, Section 8 gives our conclusion.

2 Related work

In the recent years, user demand has shifted from traditional broadcast video to dynamic on-demand live streaming and to mobile video viewing. The presence of diverse types of end-user devices and access technologies (i.e., WiFi, 2G, 3G, 4G and soon 5G) creates a very large set of possible combinations among encoding parameters (size, resolution, frame rate, format, etc.), which often results in the need for multiple streaming formats. Scalable Video Coding has been introduced for encoding media as layers in a single file [22]; it is commonly used by Content Providers in combination with adaptive bitrate streaming techniques (i.e., MPEG DASH and Apple HLS) to adapt to varying network bandwidth. Nevertheless, different encodings might be needed for different devices. For instance, Netflix encodes a single title into 120 different versions.

The progressive delocalization of video content to improve network usage (i.e., through Content Delivery Networks) is now evolving to processing functions as well (i.e., transcoding, streaming, content manipulation). Specific objectives include the minimization of backhaul cost and access delay [24], the improvement of video quality [13], and the offload of tasks from resource- and bandwidth-constrained devices. In this respect, cloud-assisted video transcoding and streaming have been investigated to adaptively adjust the streaming flow with a scalable video coding technique based on the feedback of link quality [10, 15, 27]. Placement of media servers in multiple clouds according to location principles has been considered, also taking into account bandwidth allocation [29] and power consumption [1]. These models usually do not consider bandwidth availability in the access network, though extensions to the edge should be quite straightforward. Mobile cloud systems have been considered to offload transcoding capabilities, minimizing the overall energy consumption of both mobile devices and cloud under latency constraints [28]. Progress and commercial interest in cloud technologies for media processing is witnessed by Amazon elastic media converter services.¹

One major issue when moving processing tasks at the edge is the presence of highly-variable workloads, due to time-varying usage patterns and user mobility. The intrinsic elasticity in the cloud paradigm (i.e., the ‘pay-as-you-go’ model) has been leveraged to design scalable applications, which grow and shrink processing resources according to the actual workload [11, 13, 26]. Distributed and cloud-based paradigms (e.g., the Hadoop Map-Reduce framework) have also been used to split transcoding on multiple parallel instances [17–19].

Cloud installations raise efficiency concerns when resources are not fully utilized [2]. Indeed, inconstant resource utilization is a typical behavioral pattern for mobile multimedia

¹ Amazon Elastic Transcoder: <https://aws.amazon.com/elastictranscoder/>.

systems, often with daily, weekly, or monthly periodicity. Energy-efficiency have been seldom considered for media processing at the edge, mainly as generic function of transmission [3]. Generally, energy-efficiency mechanisms consist in workload consolidation strategies, without either explicit knowledge of the applications hosted in VMs or specific support for elastic applications. Similar to our purpose, Shirayanagi et al. [23] considered the presence of backup replicas and links, and investigated through simulations how power consumption changes when varying their number. However, the focus was on the power consumption of the network alone. Li et al. [20] investigated the impact of overprovisioning (e.g., for dealing with sudden peaks of workload) on power consumption. Their strategy for workload consolidation only considered the servers, and did not take into account any network metric. In this case, they provided a real implementation, based on their iVIC virtualization infrastructure. Evaluation on violations of service-level agreements in case of consolidation was carried out by Buyya et al. [7], again through simulations. Heller et al. [16] analyzed how redundancy and guard margins influence the power consumption; their work placed VMs with the objective to minimize the power consumption of the data center network, but did not consider computing servers. Since the ground for any consolidation algorithm is the capability to move VMs between servers, Voorsluys et al. [25] studied the impact of live migration on application response time.

More recently, Beloglazov et al. [4] proposed OpenStack Neat for balancing QoS and energy consumption. Their framework monitors CPU utilization by VMs, estimates overloading/underloading conditions on each server, and uses OpenStack APIs for live-migration of VMs between servers. Their consolidation strategy is power-unaware, since only the utilization metric is used. Though power saving should stem by putting idle servers to sleep, their hardware did not support this state, so they only estimated the energy saving. Compared to our approach, this framework does not include a power monitoring framework, software-defined networking, and specific context for applications. Cima et al. [12] also proposed OpenStack extensions for energy efficiency. This work has very similar objectives with ours: an energy-efficient cloud infrastructure with enhanced API for power management. The API includes specific commands to change the power state of servers (sleep/active). The authors measured migration times and service downtime for live migration, and energy saving with power-unaware consolidation strategies similar to Neat. Our work follows a similar approach, but we also include software-defined networking for bandwidth reservation and context information for VMs, which enable better QoS management and more aggressive consolidation strategies (see Section 3).

3 An effective and efficient paradigm for elastic applications

Elastic applications can split the workload among multiple parallel processes in different VMs. They scale in and out, i.e., the number of VMs is increased or decreased when the workload changes. However, the time to deploy an additional VM is not negligible and does not match the strict latency requirements for real-time media services, as video streaming and interactive gaming. For this reason, this kind of applications are typically sized to deal with peak load and do not scale, so to avoid service disruption and delay caused by technical and administrative provisioning procedures. This results in higher operational costs and large inefficiency, because VMs are always allocated yet idle most of the time.

We introduced a novel paradigm for effective and efficient management of elastic applications at the edge in our previous work [9]. Our approach basically provides a cost-effective virtual *sleep* state which is logically equivalent to the hardware suspend-to-ram (ACPI S3

state²). The concept is quite straightforward. When the service provider deploys an elastic application, it provisions all VMs, but only activates those that are strictly necessary, while the other replicas are kept in the sleep state. Sleeping VMs are cheaper, because the cloud provider can apply aggressive power management mechanisms, based on its consolidation and energy-efficiency strategies. However, sleeping VMs return available in a very short time (indicatively, below some hundred milliseconds) so to avoid violations in service level agreement for the user's service.

Our framework is made of two parts. On the one hand, cloud users provide suitable context to distinguish between active and sleeping VMs, as well as the specification of QoS constraints (bandwidth, CPU), i.e., the application model. On the other hand, the cloud infrastructure provides all the functions needed for energy-efficient management and QoS guarantees.

3.1 Application model

Recent cloud management software includes the capability to specify meta-data and annotations for VMs. For instance, OpenStack allows the definition of properties in the form $\langle \text{key}, \text{value} \rangle$, which are associated to each VM and stored in the metadata server.³ These data are not directly interpreted by OpenStack, but can be used by external control and management tools, e.g., based on TOSCA [30] and ETSI MANO [31], which make extensive usage of metadata and annotations to drive the (semi-)automated orchestration process.

Our framework includes specific context information concerning QoS and energy efficiency. Specifically, we consider the following kinds of information for each VM:

- *service level*, related to the criticality of the service and the expected QoS (CPU load, network bandwidth);
- *availability*, indicating whether the component will be used or not in the next time-frame.

We model the above information by a colored label associated to each VM:

- *red*, for those VMs that require strict commitment on processing, memory, availability, and bandwidth requirements;
- *yellow*, for those VMs that do not operate close to their performance limit, so there is some degree of flexibility on QoS constraints;
- *green*, for unused (idle) VMs pre-provisioned for scalability.

Red labels are assigned to VMs that host the most critical software, with strict QoS constraints. As compensation for higher fares, users expect no overcommitment and no live migration to occur in this case. For instance, full CPU usage is always assumed, while network bandwidth is reserved according to the maximum declared throughput. This label may be assigned to real-time streaming services, or real-time online gaming. Yellow labels are assigned to standard VMs, for which overcommitment is acceptable. In this case, CPU usage

²The Advanced Configuration and Power Interface (ACPI) [32] is an industry-led specification that brings power management under the control of the operating system, instead of delegating to platform-specific firmware. The S3 state consists in shutting down most hardware but the system RAM, which remains powered to quickly resume the whole system to full operation.

³Metadata can be inserted when the VM is created, but can also be updated later. See: <https://wiki.openstack.org/wiki/NovalImageCreationAPI>.

may be assumed equal to a declared average load, while network reservation may use average or equivalent bandwidth values. This label may be used for non real-time transcoding agents. Finally, green labels are assigned to idle and unused VMs. They can be subject to extreme overcommitment and aggressive power-saving mechanisms, provided they become available with very low delay (i.e., below a few seconds). This label corresponds to the virtual sleep state, and should be used for spare VMs deployed by elastic applications.

In addition to labeling, we also include max/average CPU utilization and network throughput as metadata. We remark that the current classification is mostly tailored to validation of the whole concept. The number of labels and QoS parameters may be arbitrary enriched in real implementations.

3.2 Energy-efficient infrastructure

The design of our energy-efficient infrastructure improves similar initiatives which share the same objectives [4, 12, 21]. Figure 1 shows the architecture of our energy-efficient infrastructure, which includes:

- computing hardware and hypervisors, with power-saving mechanisms;
- cloud management software (CMS) that implements the Infrastructure-as-a-Service (IaaS) model;
- software-defined network (SDN), fully programmable for optimizing network paths according to bandwidth requests and with power-saving mechanisms;

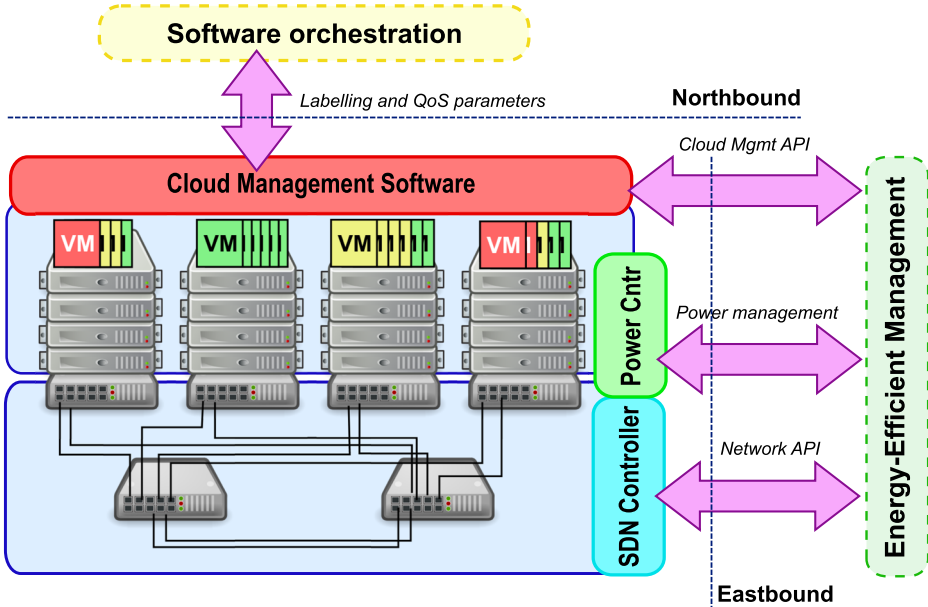


Fig. 1 Architecture of the energy-efficient infrastructure

- monitoring, which collects measurements about power consumption and CPU utilization from all servers and network devices;
- management interfaces, which allow interaction both for using and managing the infrastructure.

It provides the Infrastructure-as-a-Service (IaaS) model and exposes two kinds of interfaces: North and East. The Northbound interface is conceived to deploy applications, and provides traditional cloud APIs enhanced with QoS metadata (see Section 3.1). It could be used by human staff or by automatic orchestration engines. The Eastbound interface is used for energy-efficient control and management, including workload consolidation [8] (i.e., to reserve bandwidth, to trigger power-saving mechanisms, to migrate VMs). The same interface is also used to use the infrastructure in edge computing frameworks, as ETSI MEC [14] (i.e., for administering tenants, for connecting to external networks).

The current implementation of the architecture shown in Fig. 1 is based on open-source tools. We use OpenStack as cloud management software and OpenDayLight as network controller. This software is largely used both by the research community and in production environments, therefore they are ideal tools for conducting experimentation in relevant environments. In particular, OpenStack includes several features for supporting energy-efficient operation (see below) and it has already been used successfully by many orchestration tools. The power controller is a collection of libraries, scripts, and agents to collect power measurements and to change the power states. We can retrieve power measurements from conventional power meters by SNMP (we used power measurement outlets in our testbed) or from the IPMI interface (such data are available, for instance, from recent Intel motherboards). Power-saving mechanisms consist in two ACPI states for servers and network devices: active (ACPI S0) and sleep (ACPI S3). The power state is changed to ‘sleep’ by the power controller through a proprietary protocol; devices are resumed back to the active state by the Wake-on-Lan (WoL) protocol.

The Northbound interface is basically the OpenStack user API. We set bandwidth requirements and label color as (key,value) metadata pairs for each VM. In addition, green VMs must be put in the OpenStack PAUSED state, which is a software abstraction of ACPI S3. The PAUSED state does not save energy itself (indeed, this state is seldom used in practice); the real saving comes from workload consolidation. In more concrete terms, workload consolidation can cluster together all red and yellow VMs on the smallest number of servers, according to resource constraints. All remaining servers (including those hosting green VMs) are put in hardware ACPI S3 state; they consume very low power, but can be resumed quickly whenever an application scales and a green VM must be activated.

QoS support in OpenStack is only limited to operation of software switches. Therefore, QoS metadata in our framework are mostly conceived to be used by the SDN controller to configure physical networking equipment.

The Eastbound interface includes the cloud management API, the power management interface, and the network API. The cloud API is the standard OpenStack interface, used in admin mode to retrieve information about running VMs, state, tenant networks, VLAN ids, etc., and to trigger migration of VMs. The power management interface currently uses a mix of standard and proprietary protocols for retrieving information about current power consumption and changing the power state of servers and network devices, as already discussed above. Finally, the Network API allows programmatic configuration of the communication

infrastructure by RESTCONF [5], which provides high-level network abstraction through the Yang model.

Additional details about the implementation and expected usage for energy-aware consolidation are given in our previous work [9].

4 The testbed

The layout of the testbed we used in our experiments is depicted in Fig. 2. It is a small installation, quite reasonable for an edge installation. The size is comparable with other similar experimental setups [4, 12, 20, 25].

The infrastructure is composed of:

- OpenStack nodes: 3 compute nodes (Intel NUC6i7KYK, i7-6770HG 2.60GHz 4 cores, 4GB RAM), 1 network node, and 1 controller node (not shown in the picture);
- OpenFlow network: 4 OpenvSwitch instances running on a heterogeneous set of low-end commercial desktops (Atom, Pentium, Phenom; 2-4 cores; around 1.1GHz), heterogeneous links with 100 Mbps/1 Gbps capacity, and 1 control node running OpenDayLight (not shown);
- 1 storage node running NFS software (not shown);
- management network on a separate infrastructure.

The cloud software is the OpenStack Newton release, while for the SDN controller we used the Carbon release of OpenDayLight. We used a mix of bash scripts and simple java programs for implementing extended APIs and functionalities. Power consumption is measured by a Raritan PX3-5260R Power Data Unit, and measurements are accessible by SNMP.

According to similar works in the literature, we selected low-end hardware for our testbed. Indeed, our work leverages innovative technologies (power saving mechanisms and protocols, software-defined networking) that are not yet largely available together in commercial devices for cloud infrastructures.

5 Performance analysis

We investigated how the proposed architecture and the selected technologies affect QoS of edge applications. Our work considered two main aspects: network bandwidth reservation (Section 5.1) and live migration (Section 5.2).

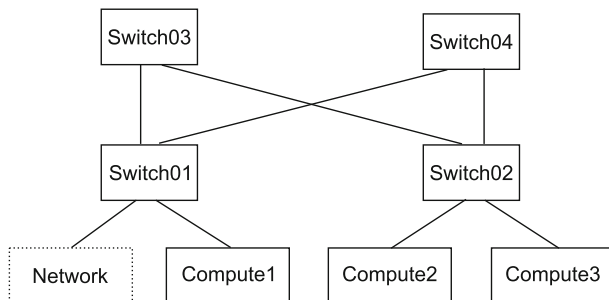


Fig. 2 Testbed topology

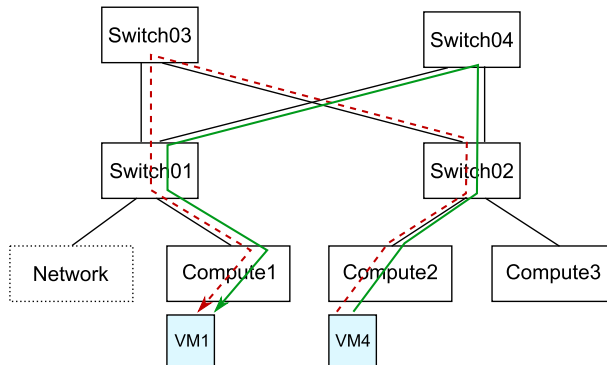


Fig. 3 Reconfiguration of network flow

5.1 Network bandwidth reservation

Our energy-efficient infrastructure allows applications to reserve bandwidth between VMs, by configuring explicit OpenFlow paths. To simplify management operations, our implementation relies on the existing *l2switch* feature in OpenDayLight to provide full connectivity among all nodes. This approach limits the number of OpenFlow rules to flows with QoS requirements (which will be mapped to higher priority traffic classes⁴), while falling back to best effort flooding behavior for ancillary traffic (e.g., DNS queries and DHCP transactions).

The *l2switch* uses controlled flooding to forward packets, so it generates a large amount of network traffic, but it is extremely resilient to failures and does not require reconfiguration in case of VM migration. Instead, explicit configuration of paths results in better and more efficient usage of the infrastructure, but needs reconfiguration in case of migration/failure that may lead to performance degradation and service disruption. Here, we investigate all these aspects and compare the two approaches.

First, we take into consideration how the reconfiguration of network flows impacts packet drops and jitters. This operation is necessary when an application is deployed and when a VM is migrated.

We generated a variable-rate UDP packet stream from VM4 to VM1 (see Fig. 3) with *iperf*,⁵ a largely used open-source tool for testing TCP, UDP, and SCTP; the usage of UDP packets enables to measure performance without the interference of any congestion-avoidance and error-recovery procedure. The original path is indicated by the red dashed line in Fig. 3, and was changed during the experiment to the green solid line by means of our enhanced Network APIs. The two paths were selected so as to generate the largest perturbation in our simple infrastructure. We measured packet rate, packet loss, and jitter, while varying the generation rate from 1 Mbps to 80 Mbps (which is the largest rate the current setup can bear without losses).

Figure 4 shows that there is no loss with low bit rate (below 1 Mbps). We see that the number of lost packets increases up to 70 (in correspondence with 50 Mbps traffic rate), then it remains quite constant. This happens because losses occur for a short time during

⁴Configuration of QoS parameters (priority queues, traffic shapers, scheduling disciplines) in the physical switches is not implemented yet, but it is already on the development roadmap.

⁵<https://iperf.fr/>.

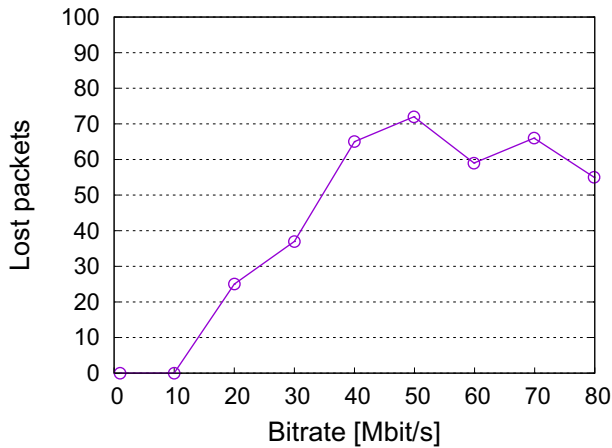


Fig. 4 Packet loss measured during flow re-configuration

the path re-configuration, which is fixed independently of the stream rate. These results are very good, since the percentage of packet loss is always below 0.14% of the total transmitted packets. We don't observe meaningful variations in packet jitters (Fig. 5).

The very good performance indexes and the low dependency on flow rate is easily explained by our implementation choice. In the time frame between the removal of the old flow and the configuration of the new one, packets are not dropped because they get flooded by the I2switch rules. However, flooding is the most reliable yet least efficient forwarding mechanism. For direct comparison between the two strategies (flooding and explicit path) in our scenario, we repeated the experiments without bandwidth reservation, hence relying on the sole operation of I2switch.

Flooding increases link utilization exponentially, so congestion happens well before unicast forwarding and causes more losses. Figure 6 shows that the number of dropped packets quickly raises one order of magnitude, with respect to unicast forwarding (the percentage

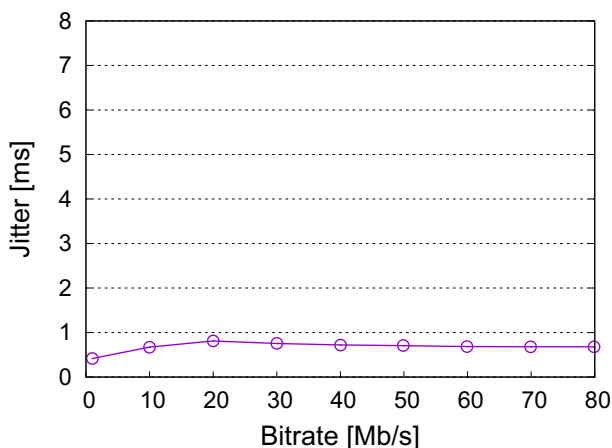


Fig. 5 Jitter measured during flow re-configuration

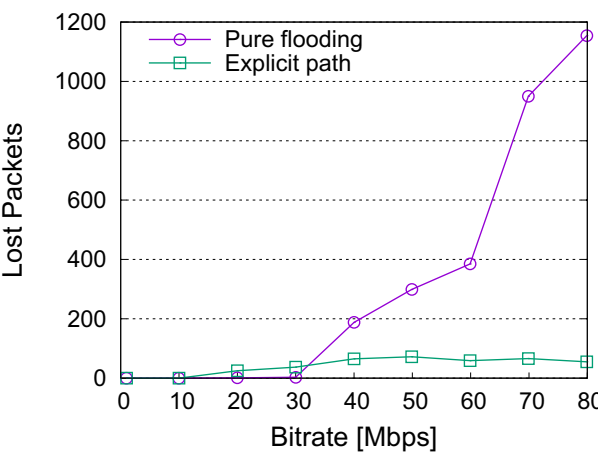


Fig. 6 Comparison of packet loss between flooding (l2switch behavior) and explicit path configuration (bandwidth reservation)

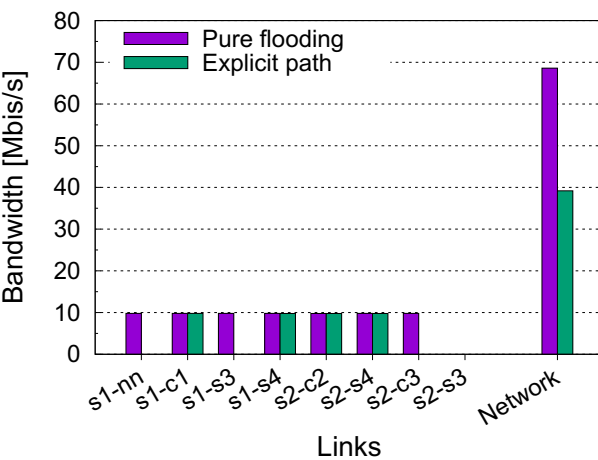


Fig. 7 Comparison of bandwidth usage between flooding and explicit path configuration. Links are identified by their peer nodes: NN=Network Node; C=Compute; S=Switch

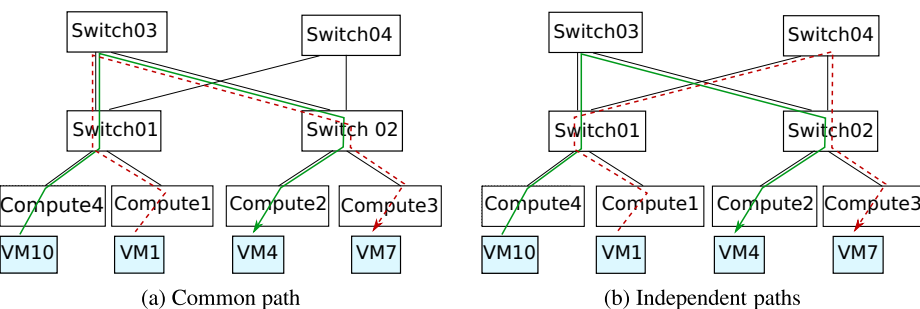


Fig. 8 Alternative forwarding strategies for packet streams exchanged by different peer VMs

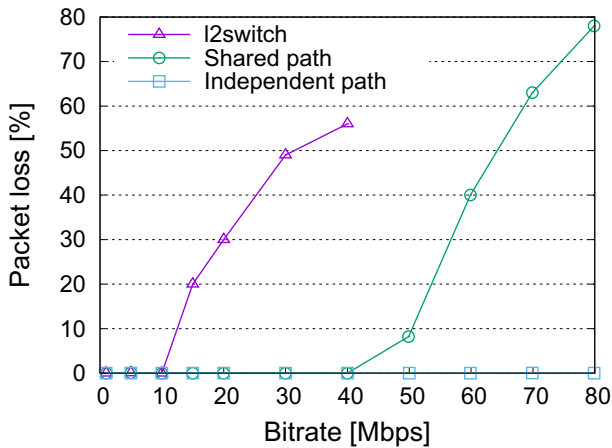


Fig. 9 Packet loss under different forwarding strategies

exceeds 1.2% for 80 Mbps). In addition, the packet stream is transmitted on all links with flooding, hence largely increasing network congestion (total utilization almost double even in our very simple topology, see Fig. 7).

Finally, we report additional measurements that show how software-defined networking enables effective usage of the whole bandwidth in our framework. We deployed 4 VMs; constant-rate traffic streams are exchanged between VM10–VM4 (green solid line) and between VM1–VM7 (dashed red line),⁶ as shown in Fig. 8. We consider three different forwarding mechanisms:

- i) l2switch behavior, with controlled flooding;
- ii) shared path, by configuring a common path for all traffic streams between Switch01 and Switch02, without considering bandwidth requirements (see Fig. 8a); this scenario is representative of legacy technologies based on the spanning tree protocol and address learning;
- iii) independent paths, by configuring separate flows for each traffic stream (see Fig. 8b), so that bandwidth requirements are matched with link capacity.

Similar to the previous experiment, we selected the placement of VMs so to address the worst-case scenario. Again, we varied the generation rate from 1 Mbps to 80 Mbps.

Figure 9 shows the packet loss for the flow between VM10–VM4 with different forwarding behaviors (data for the other flow are equivalent). As expected, relying on the built-in flooding mechanism of the l2switch OpenDayLight feature is the worst option; we stopped at 40 Mbps because it was not worth increasing the rate further. Configuring a common flow works well until the total bandwidth exceeds the link capacity (i.e., below 40 Mbps), then packets start being dropped systematically because not enough resources are available to bear them. By exploiting the whole topology with independent and dedicated flows, performances are equivalent to having a single stream (packet losses are always below 0.02%). To achieve guaranteed performances under all possible conditions, the allocation strategy must be paired with an access control that denies additional requests when there is no more available link capacity.

⁶In this case, we deployed an additional server instead of the network node.

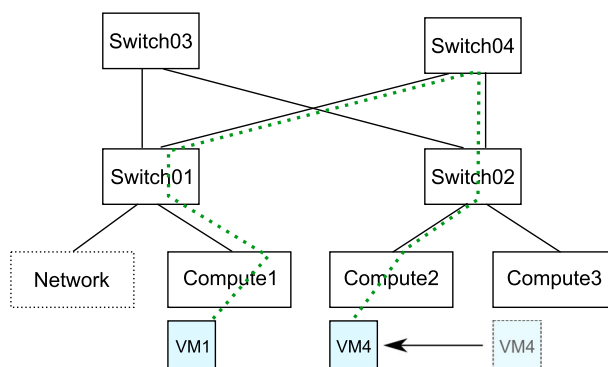


Fig. 10 Live migration of VM4 (semi-transparent VM icon indicates the previous position)

5.2 Live migration

Live migration of VMs is an iterative process that copies the guest memory from the source VM to the destination VM; the specific challenge is the minimization of the service downtime. The impact of live migration on service continuity was already investigated [25], but here we want to carry out additional measurements in our specific framework. We consider two aspects separately: impact on network traffic and impact on service continuity. In both cases, we used the setup shown in Fig. 10.

Impact on network traffic is evaluated in terms of packet drops and jitters, while alternatively migrating the transmitter and the receiver. Similar to the experiments in the previous Section, we increment the generation rate from 1 Mbps to 80 Mbps.

Figure 11a shows that packet loss is rather limited when the transmitter is migrated. This is easy to understand: during the migration, the transmitter is paused for a while and then resumed on the destination server; therefore, the stream is suspended during the migration and the few losses are due to reconfiguration of forwarding rules after the migration. Instead, packet loss when the receiver is migrated is one order of magnitude greater (Fig. 11b): since the transmitter is unaware of the migration, it continues streaming packets, which are therefore lost because the receiver is unavailable. We didn't notice any relevant variations in jitters in both cases.

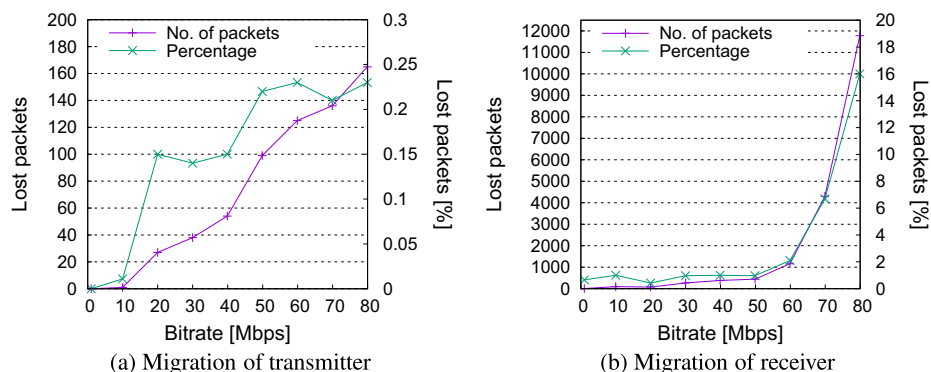


Fig. 11 Packet loss during live migration of the peers of a traffic stream

Table 1 Relevant parameters of the transcoding application

Parameter	Original value	Final value
Video length	ca 22m 52s	Same
Transcoding tool	ffmpeg 3.4.1	N/A
Video stream	h264 (native)	mpeg4 (native)
Audio stream	aac (native)	mp3 (libmp3lame)
Container format	mp4	avi
Transcoding time (w/o migration)	N/A	ca 4m 6s

For service continuity, we used a real multimedia application. We chose video transcoding, since it needs high computing load and has deterministic execution times. Table 1 lists the main parameters for our transcoding jobs. We measured how the processing time changes while varying the main operational parameters for migration; we also looked at the traffic generated on the control network.

Before coming to the results, we must briefly describe the main parameters that control live migration in our framework. Our OpenStack installation makes use of the KVM/QEMU hypervisor. In this environment, the migration process is governed by three main settings:⁷ live migration completion timeout, live migration progress timeout, and instance downtime. The last setting is the most relevant for QoS. There are 3 parameters that can be tuned:

- i) `live_migration_downtime` sets the maximum permitted downtime for a live migration. The default is 500 milliseconds.
- ii) `live_migration_downtime_steps` sets the total number of adjustment steps until `live_migration_downtime` is reached. The default is 10 steps.
- iii) `live_migration_downtime_delay` sets the time interval between two adjustment steps. The default is 75 seconds.

Live migration copies the instance's memory from the source to the destination hypervisor without stopping the instance. It works in an iterative way, by copying over and over again pages that are written by the instance in the meanwhile (and marked as *dirty* pages). To avoid looping forever in case of frequently memory writes, the instance is paused for a short time every `live_migration_downtime_delay` per GB of memory, so that the remaining few pages can be copied without interference from memory writes. This interval is initially set to `live_migration_downtime/live_migration_downtime_steps` (50 ms with default values). If the memory copy cannot be completed within this interval, the instance is resumed and the process repeats from beginning. The procedure increments the downtime interval linearly at successive iterations until the maximum permitted value (`live_migration_downtime`) is reached; if a clean copy of the memory is not completed, the migration fails.

If `live_migration_downtime` is small, live migration is attempted several times, and it may eventually fail. Instead, a large value may cause excessive downtime to the application. Therefore, we investigated how this parameter affects both migration and service times.

Figure 12 shows the traffic generated on the control network during the migration. Two different scenarios are considered: migration of one single VM and concurrent migration of

⁷OpenStack documentation – Configure live migrations. URL: <https://docs.openstack.org/nova/pike/admin/configuring-migrations.html>.

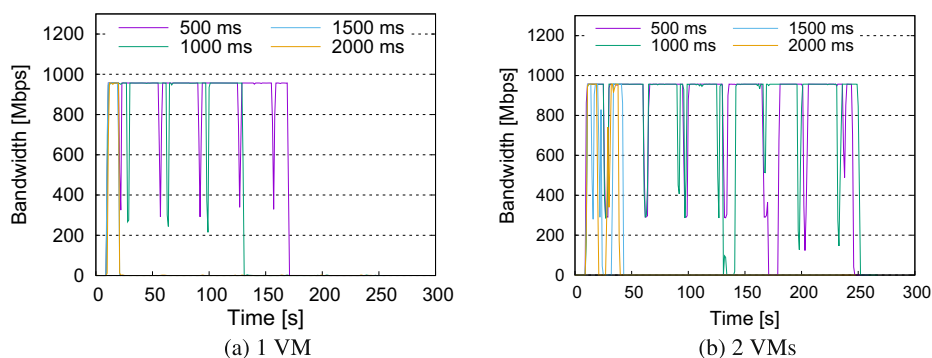


Fig. 12 Traffic generated in the control network during live migration, while changing the `live_migration_downtime` parameter from 500 ms to 2000 ms. The other parameters are set as: `live_migration_downtime_delay=10 s`; `live_migration_downtime_steps=10`

two VMs. The longer the utilization peak in the graphs, the longer the migration. Migration ends when bandwidth utilization falls close to zero. We note periodic drops during the migration, due to the step-incremental behavior discussed above. As expected, when multiple migrations are active simultaneously, the total migration time is longer. We noted that requests for migration are queued and carried out sequentially by OpenStack, rather than in parallel.

Note that a shorter max downtime value (e.g., 500 ms) leads to longer migrations, hence wasting bandwidth and causing periodic incremental service disruptions. This raises the suspect that, overall, small downtime values might lead to longer service disruption and unavailability than larger values, even if that looks counterintuitive.

The above impression is confirmed by analyzing the total processing times for video transcoding (see Fig. 13). We measured a small increase in the service processing time, about 4 s (1.6%), while using the two smaller downtime values. We also see the great reduction in total migration times when using longer downtime values. From this evaluation, we conclude that setting longer downtimes is beneficial both from service and migration performances, hence we fixed this parameters to 2000 ms in all successive trials.

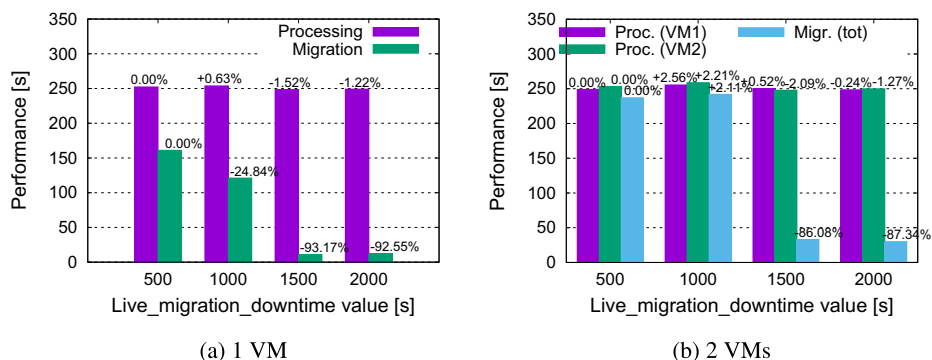


Fig. 13 Processing vs migration times for live migration of a single VM or 2 VMs

Table 2 Placement of VMs for different consolidation strategies

Consolidation strategy	Placement decision		
	Compute1	Compute2	Compute3
1. Max saving	All VMs (VM1–6)	–	–
2. Balanced	3 VMs (VM1,3,5)	3 VMs (VM2,4,6)	–
3. Max performance	2 VMs (VM1,4)	2 VMs (VM2,5)	2 VMs (VM3,6)

6 Energy efficiency: performance vs power consumption

Given the antithetical goals of energy efficiency and QoS, it is natural to wonder how they are balanced in our paradigm. Indeed, the two aspects are often investigated separately, and their correlation is not studied. To this aim, we consider how consolidation affects the overall efficiency; the latter is given by the ratio between performance and power consumption.

We suppose to have 6 VMs, and we suppose 3 different placement decisions, corresponding to more or less aggressive consolidation strategies. The placement of VMs is described in Table 2.

We evaluated with real data how power consumption and performances change for each strategy. We used the stress-ng⁸ tool to incrementally increase the CPU load for each VM. We consider 100% as full utilization of one VMs, so the total load for six VMs equals to 600%; the incremental step is fixed at 25%. We increment the load for each VM in numerical order, i.e., first we increase the load for VM1 and, when it is full (100%), we start with VM2, and so on until the last VM (VM6) is fully loaded.

We show power consumption and performances for the three strategies in Fig. 14. We see that a large power consumption gap is present between Strategy 1 and Strategy 2 (around 40% increase), while the difference is limited between Strategy 2 and Strategy 3 (Fig. 14a). This is obvious if we consider the network topology (Fig. 2): by placing VMs on Compute2, we need to wake up two additional switches (Switch02 and Switch03), which are not necessary for Strategy 1, whereas no additional network devices are necessary when switching from Strategy 2 to Strategy 3.

Even though Strategy 1 achieves greater energy saving, we must also take performance into account for fair comparison. To this aim, we consider number of operations per second, as reported by stress-ng metrics for full-load experiments (i.e., all VMs running at 100% CPU). Though such metric is not a reliable performance indicator and has not been designed to be used for scientifically accurate benchmarking, it is enough to get a notional rough comparison of performance between different set-ups. We note that performances are almost constant while running 1, 2, or 3 VMs concurrently on the same hypervisor, but they significantly degrade for 6 VMs (Fig. 14b). This is not surprisingly, since each processor on the compute nodes has 4 cores with hyper-threading; though they appear as 8 vCPUs to the OS, hyper-threading is not equivalent to having additional real cores.

For better evaluation, we compare in Fig. 15 the variation of three relevant indexes for the different strategies, arbitrarily assuming Strategy 1 as base reference (100%): performance,

⁸Stress-ng is a tool that stresses a computer system in various selectable ways. Stress-ng has a wide range of CPU-specific stress tests that exercise floating point, integer, bit manipulation and control flow. URL: <http://kernel.ubuntu.com/~cking/stress-ng/>.

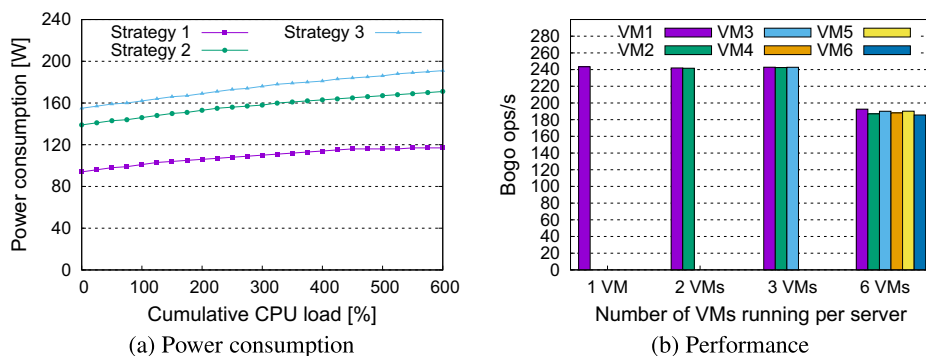


Fig. 14 Comparison of power consumption and performance for three consolidation strategies

power, and overall efficiency. We consider the number of total operations per second for “performance,” the “power” consumption of the whole infrastructure, and the ratio between number of operations and power consumption for “efficiency.” We take into consideration three level of CPU utilization: 100% (full load), 75% (high load), 50% (medium load).

Figure 15 shows that, for full-load utilization, Strategy 2 and 3 have almost identical performance (+28% with respect to Strategy 1), but different power profiles (+35% and +44% increase). Strategy 1 is undoubtedly the most efficient solution (+5%, +11% than other Strategies), though this comes at the cost of worse performances. We can say that Strategy 2 is better than Strategy 3, since we have higher efficiency and lower power consumption with the same performance level. Instead, comparison between Strategy 1 and Strategy 2 is not straightforward, because it depends on the overall optimization objectives (which may lean towards performance or energy saving). In general, Strategy 1 is preferable, since the energy saving is larger than performance penalty, hence efficiency is higher with respect to the other scenarios.

When VMs need less CPU time (i.e., lower utilization level), there is more room for overcommitment. Indeed, the performance increment is limited to 15% with 75% CPU load and negligible with 50% CPU load. Since the power increment is comparable for all CPU loads, this turns into greater loss of efficiency from Strategy 1 to Strategy 3.

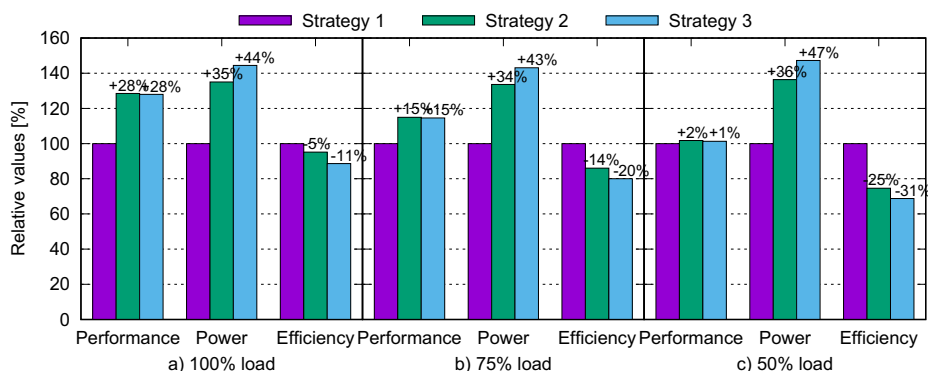


Fig. 15 Comparison of performance, power, and overall efficiency for the three consolidation strategies, under different utilization factors

We conclude by remarking that more aggressive consolidation (Strategy 1) is the optimal choice when VMs are not 100% loaded. Instead, when VMs are fully loaded, less aggressive consolidation brings larger performance gains, which motivates a slight loss of efficiency. In general, less aggressive consolidation is better suited for latency-sensitive and mission-critical applications, where performance should prevail over power efficiency. This observation should be taken into account while designing consolidation algorithms; the labeling scheme used in our consolidation strategy enables to easily take into account the different requirements, hence supporting the definition of optimal consolidation plans tailoring the need of different classes of applications.

7 Consolidation and energy saving

The last part of our work is devoted to evaluation of energy saving with a sample application. To this purpose, we developed a very simple elastic transcoding application, made of one control master and several slaves. The master loads video files and sends them to the slaves; we name it “dispatcher.” The slaves are indicated as “workers;” they transcode the video files in parallel, one on each worker. This is representative of a large set of applications, which could be implemented with big data frameworks (e.g., Hadoop, Spark). Since the main purpose is the evaluation of efficiency, we do not consider more complex actions as video queuing or file splitting in multiple segments.

We allocated one VM for the dispatcher and 9 VMs for workers (worker1, worker2, ..., worker9); the dispatcher is considered a critical service (“red” label), while workers are paused/unpaused depending on the current workload. The scaling strategy is therefore very simple: every new transcoding request is assigned to the first available worker (ordered by their identifier). Each VM has 1 vCPU and 2 GB RAM. For simplicity, we do not overcommit resources, i.e., the maximum number of VM per server is limited to 4 (since each server has four physical cores).

We also developed a simple energy management and consolidation strategy, which makes use of the Eastbound interface. It periodically puts idle servers in the sleep state, and wakes

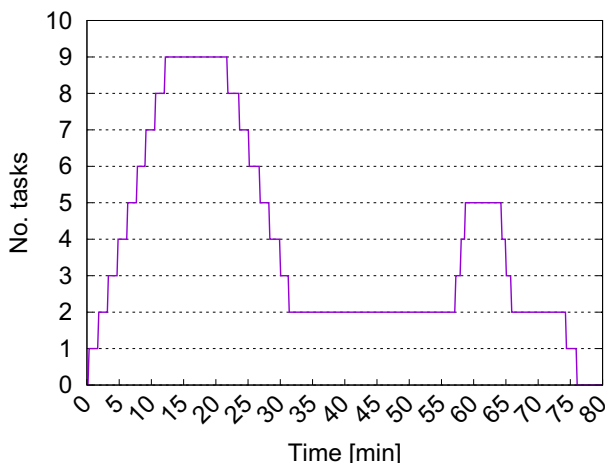


Fig. 16 Workload profile with two peaks of requests

up them when a request to ‘unpause’ VMs is received. In addition, it performs consolidation, by clustering VMs on the smallest number of servers, minimizing the overall power consumption of computing servers and network switches. Given the testbed topology and the fact that the network node never sleeps because of its specific function, the consolidation results in placement of VM on the leftmost servers, so to minimize the number of active network devices.

Initially, we generated 9 transcoding requests, one every 90 seconds; the same video was submitted by the first 7 requests, which takes around 21 minutes to complete, and a longer video was submitted by the last two requests (around 63 minutes transcoding time). Then, 57 minutes after the beginning of the simulation, we submitted other 3 shorter videos (with transcoding time around 7 minutes), one every 45 seconds. Figure 16 shows the evolution in time of the workload. At each time instant, the curve indicates the number of active transcoding jobs; therefore, each step up corresponds to the arrival of a new video and each drop to completion of a transcoding request. The profile includes a 30-minutes traffic peak, 30-minutes steady conditions with two jobs, and a short 10-minutes spike. This situation is representative of sudden variations in workload, which are more difficult to tackle than steady state, and will be used to estimate the effectiveness of the consolidation algorithm under different parameters.

Without consolidation (baseline scenario), all devices remain active all the time, even if they are idle. The power consumption of each device and the whole system is depicted in Fig. 17. We see a clear dependency on the workload: power increases when more workers are transcoding videos.

A typical consolidation algorithm (legacy scenario) improves the efficiency, by putting idle devices in sleep mode. However, ignoring the evolving context (i.e., VM usage) limits the effectiveness of such algorithm. As a matter of fact, the 10 VMs are deployed in 3 servers, due to capacity constraints, which therefore remain always active. Switch04 can be put to sleep: it is provided for redundancy and load sharing, but our simple application has very low requirements on network bandwidth, so all flows can be consolidated on a single core switch. Figure 18 shows that with workload consolidation the power consumption is lower than the baseline scenario; the overall shape is the same, but about 70 W are saved by running Switch04 in sleep mode.

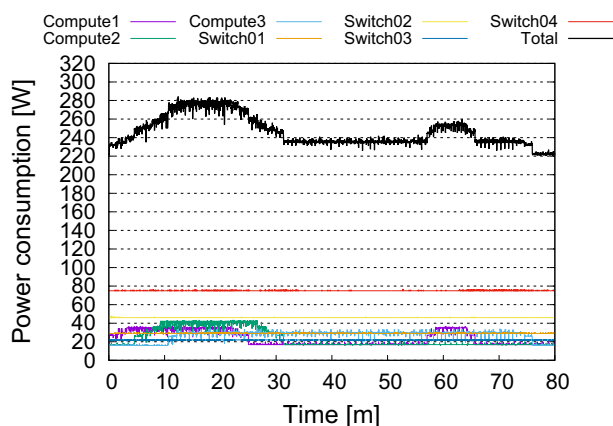


Fig. 17 Power consumption without consolidation (baseline scenario)

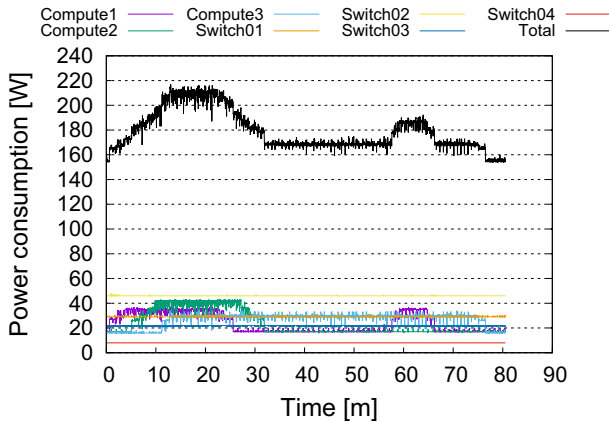


Fig. 18 Power consumption with consolidation that does not use context information (legacy scenario)

We now consider the behavior of our consolidation algorithm, improved with context information. Figure 19 shows how VMs are placed during the experiment. Before discussing the behavior of the consolidation algorithm, we point out that, just for the sake of readability, we had to down-sampling the data, so some transitions might not be present in the picture. The simple video transcoding application allocates a new job to the free worker with the lowest identifier; according, the starting placement was chosen based on the consideration that lower-identifier workers are likely to have been used more recently. As soon as new jobs are allocated, workers incrementally switch from green to yellow. Nothing changes in placement, until the first migration occurs (see the blue big asterisks in the top part of the graph). We see a first shift of W8 and W9 from Compute3 to Compute1 after 26 minutes from the beginning, corresponding to the movement of idle W1 and W2 to Compute3, which enables to put Compute3 to sleep. A second migration occurs after 5 minutes, when a single

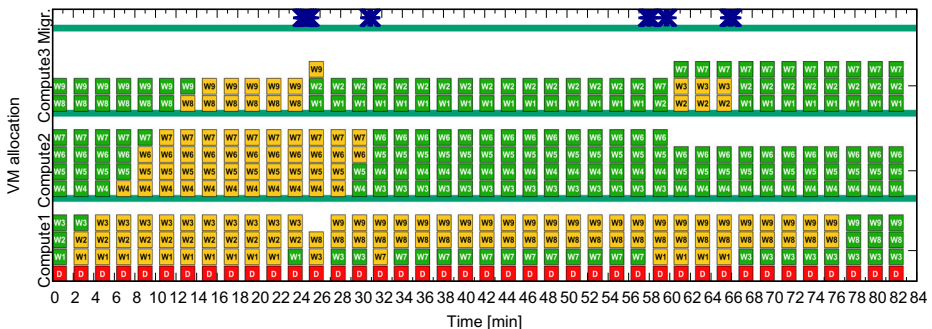


Fig. 19 Placement of VMs/workers to compute nodes and migrations (D=Dispatcher, Wn=Worker(n)). The box color represents the VM's QoS label: red (max performance guaranteed, avoid migrations), yellow (average performance guaranteed, may migrate), green (paused, sleep mode for the hypervisor is allowed). Asterisks show the time slots where migrations of VMs were in progress

worker remains active on Compute2 (namely W7): exchanging the placement of W7 and W3 permits to put to sleep Compute2 and all unused network devices. Unfortunately, W7 ends just after this consolidation is completed, hence this operation does not bring appreciable improvement (Compute2 would have been put in sleep mode anyway after completion of W7). In such situation, estimation and forecasts on future workload may further improve the consolidation algorithm. W1 is migrated back after 60 minutes from the beginning, in correspondence with the second workload peak. W2 and W3 are used after a while, but they are not migrated because Compute1 is already full and migration on Compute2 would not bring any advantage in terms of power consumption.

With reference to Fig. 19, intuitively, the longer the time intervals with all “green” boxes, the larger energy savings. This rough observation is confirmed by measurements on power consumption in Fig. 20a. Two peaks of power are present, but with larger variations than in the baseline scenario (Fig. 17). The peak power is the same as the consolidation scenario with no context information (Fig. 18), because all VMs are used in this case; however, the average consumption is always lower because sometimes some hypervisors only host idle (PAUSED) VMs and can be put to sleep.

Figure 20b shows the bandwidth usage on the control network, i.e., the main cost for live migration. We have intense but very short peaks, given the tuning for the migration parameters (see Section 5.2). It is easy to relate these peaks with the steps in power consumption (Fig. 20a) and the allocation decisions (top asterisks in Fig. 19).

We used a very simple consolidation strategy to evaluate the concept and the effectiveness of our implementation. It returns optimization decisions in real time, also given the size of the testbed. However, when more complex algorithms and models are applied to large topologies, finding an optimal consolidation plan may become cumbersome. To keep into account this limitation, we repeated the experiment while applying consolidation at the following fixed time intervals: 10 minutes, 20 minutes, 30 minutes, and 40 minutes. With longer intervals, nodes are expected to remain active for a longer time when they are idle, hence reducing the overall efficiency. Clearly, the effectiveness of the consolidation strategy largely depends on the specific instant it is started: if the algorithm is run just before the workload starts decreasing, it is necessary to wait until the next interval before making

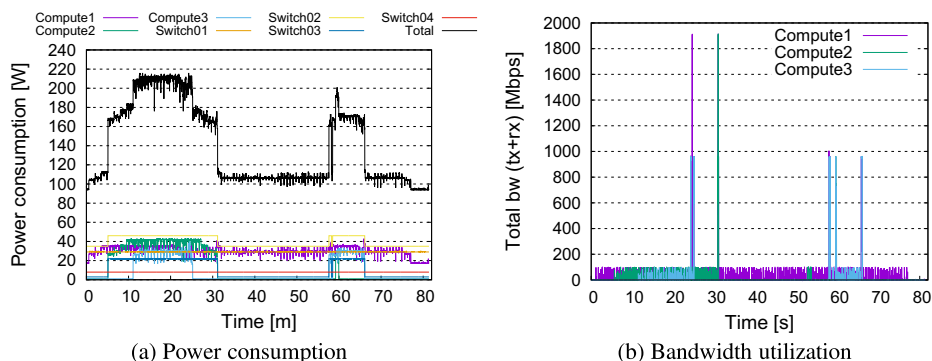


Fig. 20 Consolidation with context information. Power consumption is reported both for individual devices and the whole system; bandwidth utilization reports packet traffic on the control network due to migrations. Consolidation was performed instantly at each arrival/departure event

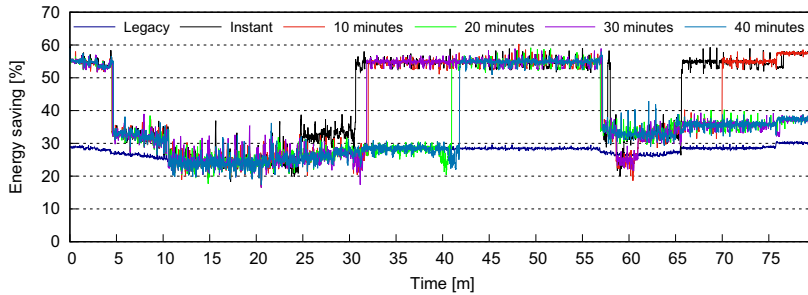


Fig. 21 Percent energy saving achieved with different optimization intervals

effective consolidation. For example, this event happens with the 20-minutes interval, where the overall behavior is very similar to the 40-minutes interval. The best optimization interval depends on the degree of variations of the workload (i.e., the slopes of the profile), hence it should be estimated for each specific scenario, maybe dynamically with weekly, monthly, season, or yearly periodicity. Nevertheless, our analysis is enough to point out the main effects that will happen in real environments.

For the sake of brevity, we omit specific graphs for each time interval (corresponding to the information plotted in Fig. 20), and directly show the comparison of power saving in Fig. 21 for the whole period under consideration (about 80 minutes). Power saving is computed with respect to the baseline scenario (Fig. 17), i.e., when no consolidation is applied.

As previously discussed, legacy consolidation (i.e., without context information) has the same saving than our method (around 25%) only when all resources are used. In all other cases, the usage of context information allows far larger saving, even almost doubling the legacy scenario when most VMs are idle.

Though most of the time the graphs overlap, it is rather clear that larger savings are achieved by shorter optimization intervals. For example, 10-minutes optimization performs almost the same as instant optimization (which represents the upper bound); 30-minutes optimization behaves the same as 10-minutes between 30 and 40 minutes due to the favorable time it is run, but it will not capture the ramp-down of the second workload spike until 90 minutes (not shown in the graph). 20- and 40-minutes optimization perform almost identically, but this is due to the timing of the two workload profiles; in general, we would expect saving for 20-minutes optimization to position between those for 30- and 40-minutes. In general, more migrations happen with shorter optimization intervals; this does not hold for the 20-minutes interval, due to the timing of the workload profile.

Finally, to better appreciate the quantitative differences between the different consolidation algorithms and intervals, we compare energy saving in different time windows in Fig. 22, by dividing the whole experiment period in 10-seconds intervals. We remark that measures come from different realizations of the same experiment, so some variance is inborn in data, namely the values for power consumption should be compared with a tolerance of about 1 percentage point. According to the trend shown in Fig. 22, we note that optimal performances are achieved during steady-state periods (e.g., from 40 to 60 minutes after beginning of the experiments), while penalties for longer optimization intervals are predominant after ramp-downs of the workload profile (i.e., from 25 to 40 minutes and from 65 to 80 minutes after the beginning of the experiments).

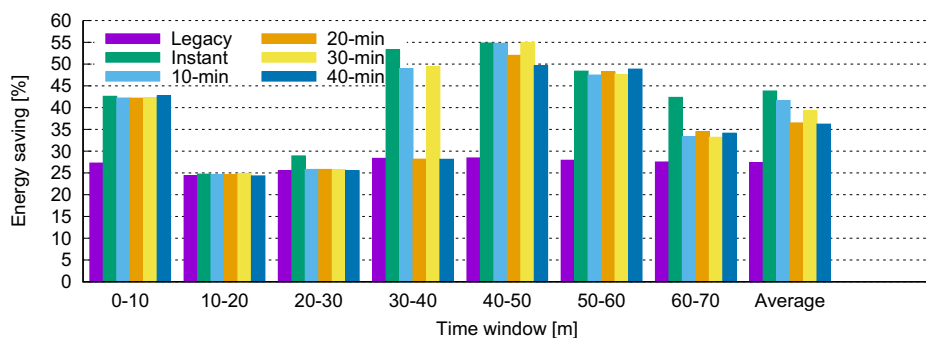


Fig. 22 Comparison of energy saving in different time windows during the experiments

8 Conclusion

The deployment of elastic multimedia applications at the edge necessarily relies on the capability of provisioning virtual resources in nearly real-time. To this aim, we have introduced a IaaS paradigm that provisions VMs in advance, while keeping their impact on operational costs negligible. The main enabler of our concept is an energy-efficient infrastructure that extends existing cloud management software with power-management and QoS capabilities.

In this paper, we have evaluated performance and power consumption in a real testbed. Our analyses has showed that the proposed edge infrastructure can effectively reserve bandwidth, support live migration, and save more energy than conventional workload consolidation. Further investigation will consider pricing models for active and frozen resources, which reflect energy saving into suitable cost reduction for cloud users, pursuing an optimal balance between revenues for infrastructure providers and economical convenience for edge users.

Acknowledgments This work was supported in part by the European Commission under the projects ARCADIA (contract no. 645372) and MATILDA (contract no. 761898).

References

1. Baker T, Asim M, Tawfik H, Aldawsari B, Buyya R (2017) An energy-aware service composition algorithm for multiple cloud-based IoT applications. *J Netw Comput Appl* 89(1):96–108
2. Barroso LA, Hölzle U (2007) The case for energy-proportional computing. *Computer* 40(12):33–37
3. Beck MT, Feld S, Fichtner A, Popien CL, Schimper T (2015) ME- VoLTE: network functions for energy-efficient video transcoding at the mobile edge. In: *Proc. of 18th Intl. conf on intelligence in next generation networks (ICIN)*. Paris
4. Beloglazov A, Buyya R (2015) OpenStack Neat: a framework for dynamic and energy-efficient consolidation of virtual machines in OpenStack clouds. *Concurr Comput Pract Exper* 27(5):1310–1333
5. Bierman A, Bjorklund M, Watsen K (2017) RESTCONF protocol. RFC 8040. <https://tools.ietf.org/html/rfc8040>
6. Bolla R, Repetto M (2013) A comprehensive tutorial for mobility management in data networks. *Commun Surv Tutor IEEE PP*(99):1–22
7. Buyya R, Beloglazov A, Abawajy J (2010) Energy-efficient management of data center resources for cloud computing: a vision, architectural elements, and open challenges. In: *Proceedings of the 2010 international conference on parallel and distributed processing techniques and applications (PDPTA 2010)*. Las Vegas

8. Carrega A, Repetto M (2017) Energy-aware consolidation scheme for data center cloud applications. In: 8th International workshop on softwarized infrastructures for 5G and fog computing (SoftS 2017). Genoa
9. Carrega A, Portomauro G, Repetto M, Robino G (2018) OpenStack extensions for QoS and energy efficiency in edge computing. In: 3rd IEEE International conference on fog and edge mobile computing (FMEC 2018). Barcelona, <http://www.reti.dist.unige.it/matteo/papers/qosvsee.pdf>, to appear
10. Chen M (2012) AMVSC: a framework of adaptive mobile video streaming in the cloud. In: Proc. of IEEE global communications conference (GLOBECOM). Anaheim, pp 2042–2047
11. Cheng R, Wu W, Lou Y, Chen Y (2014) A cloud-based transcoding framework for real-time mobile video conferencing system. In: Proc. of 2nd IEEE intl. conf.on mobile cloud computing, services and engineering (MobileCloud). London
12. Cima V, Grazioli B, Murphy S, Bohnert T (2015) Adding energy efficiency to Openstack. In: Sustainable internet and ICT for sustainability (SustainIT). Madrid, pp 1–8
13. Dutta S, Taleb T, Frangoudis PA, Ksentini A (2016) On-the-fly QoE-aware transcoding in the mobile edge. In: Proc of IEEE global communications conference (GLOBECOM). Washington, DC
14. ETSI (2016) Mobile edge computing (MEC); framework and reference architecture. ETSI GS MEC 003. http://www.etsi.org/deliver/etsi_gs/MEC/001_099/003/01.01.01_60/gs_MEC003v010101p.pdf, v1.1.1
15. Fajardo JO, Taboada I, Liberal F (2015) Improving content delivery efficiency through multi-layer mobile edge adaptation. IEEE Netw 29(6):40–46
16. Heller B, Seetharaman S, Mahadevan P, Yiakoumis Y, Sharma P, Banerjee S, McKeown N (2010) Elastictree: saving energy in data center networks. In: Proc. of the 7th USENIX conf. on net. sys. des. and impl., USENIX Association, Berkeley, CA, USA, NSDI, pp 17–17
17. Jokhio F, Deneke T, Lafond S, Lilius J (2012) Bit rate reduction video transcoding with distributed computing. In: Proc. of 20th Intl. conf. on parallel, distributed and network-based processing (PDP). Garching, pp 206–212
18. Kim M, Han S, Cui Y, Lee H, Cho H, Hwang S (2014) CloudDMSS: robust hadoop-based multimedia streaming service architecture for a cloud computing environment. Clust Comput 17(3):605–628
19. Lao F, Zhang X, Guo Z (2012) Parallelizing video transcoding using map-reduce-based cloud computing. In: Proc. of IEEE Intl symposium on circuits and systems (ISCAS). Seoul
20. Li B, Li J, Huai J, Wo T, Li Q, Zhong L (2009) Enacloud: an energy-saving application live placement approach for cloud computing environments. In: IEEE International conference on cloud computing (CLOUD '09). Bangalore, pp 17–24
21. Rossigneux F, Gelas JP, Lefèvre L, de Asunção MD (2014) A generic and extensible framework for monitoring energy consumption of openstack clouds. In: 4th IEEE International conference on sustainable computing and communications (SustainCom). Sydney
22. Schwarz H, Marpe D, Wiegand T (2007) Overview of the scalable video coding extension of the H.264/AVC standard. IEEE Trans Circ Syst Video Technol 17(9):1103–1120
23. Shirayanagi H, Yamada H, Kono K (2012) Honeyguide: a VM migration-aware network topology for saving energy consumption in data center networks. In: IEEE Symposium on computers and communications (ISCC). Cappadocia, pp 460–467
24. Tran TX, Pandey P, Hajisami A, Pompili D (2016) Collaborative multi-bitrate video caching and processing in mobile-edge computing networks. arXiv:1612.01436
25. Voorsluys W, Broberg J, Venugopal S, Buyya R (2009) Cost of virtual machine live migration in clouds: a performance evaluation. In: Proceedings of the 1st international conference on cloud computing (CloudCom '09). LNCS: Lecture Notes In Computer Science, pp 254–265
26. Wang F, Liu J, Chen M (2012) CALMS: cloud-assisted live media streaming for globalized demands with time/region diversities. In: Proc. of IEEE conference on computer communications (INFOCOM). Orlando, pp 199–207
27. Wu Y, Wu C, Li B, Lau FCM, 2013 vSkyConf: cloud-assisted multiparty mobile video conferencing. In: Proc. of the 2nd ACM SIGCOMM workshop on mobile cloud computing. Hong Kong, pp 33–38
28. Zhang W, Wen Y, Chen HH (2014) Toward transcoding as a service: energy-efficient offloading policy for green mobile cloud. IEEE Netw 28(6):67–73
29. Zhao Y, Jiang H, Zhou K, Huang Z, Huang P (2014) Meeting service level agreement cost-effectively for video-on-demand applications in the cloud. In: Proc. of IEEE conf. on computer communications (INFOCOM). Toronto, pp 298–306
30. (2013) Topology and orchestration specification for cloud applications. OASIS Standard. <http://docs.oasis-open.org/tosca/TOSCA/v1.0/os/TOSCA-v1.0-os.pdf>, version 1.0
31. (2014) Network functions virtualisation (nfv); management and orchestration. ETSI GS NFV-MAN 001. http://www.etsi.org/deliver/etsi_gs/NFV-MAN/001_099/001/01.01.01_60/gs_nfv-man001v010101p.pdf, v1.1.1
32. (2017) ACPI Specification Version 6.2 (Errata A). Unified Extensible Firmware Interface Forum

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Alessandro Carrega is currently a research associate at CNIT. He took part in the activities of many national and European projects and he is an active reviewer for many different international journals and conferences (IEEE and ACM). He has co-authored several papers in international conference proceedings. His main research topics are on networking (energy-aware, performance optimization, and virtualization), software routers, NFV and SDN (in particular, OpenFlow).



Giancarlo Portomauro is research fellow at Telecommunications Engineering, and Naval Architecture (DITEN) at the University of Genoa. He is currently working on projects related to cloud computing and energy-efficient networking. His main activities concerns: emulation of commutation of on board satellite systems, reliable QoS network satellite for multimedia applications, software for the satellite emulation system, installation and training for video conference tools for remote training and instruments access, design and realization of event simulator for heterogeneous packet switching networks.



Matteo Repetto is a research associate at CNIT. He has been teaching many courses in telecommunication networks and network security. He has been involved in many different national and European research projects in the networking area. His current research interests include pervasive communications and mobility management, energy-efficient networking, software-defined networking and network function virtualization, cloud computing.



Giorgio Robino is senior software developer/designer and project manager, with experience in many different fields: factory automation, mobile telecom, banking, web apps, e-commerce, cloud computing. He has been involved in several research projects on cloud computing and energy efficiency (ARCADIA and FIWARE-Lab). He is currently awarded a research grant at the National Research Council (CNR). His current research interests also include Conversational AI, natural language understanding, voice user interfaces.